

普及组初赛模拟 27

一、单选题

1. 计算机存储和处理数据的基本单位是 ()
A、*Bit*
B、*Byte*
C、*KB*
D、*GB*
2. 用两个字节最多能编出多少个不同的码 ()
A. 256 B. 65536 C. 32768 D. 2147483648
3. 具有 18 个元素的有序数组, 其中第一个元素下标为 1, 现进行二分查找, 则查找到下标为 3 的元素, 比较序列的下标依次为 ()
A. 1, 2, 3 B. 9, 5, 2, 3 C. 9, 5, 3 D. 9, 4, 2, 3
4. 设二叉树的后序序列为 *DBFEGCA*, 中序序列为 *DBAFECG*, 则它的先序遍历为 ()
A、*ABCDFEG*
B、*ABDCFEG*
C、*ABCDEF G*
D、*ABDCEFG*
5. 设输入序列为 1、2、3、4、5、6, 则通过栈的作用后可以得到的输出序列为 ()
A. 5, 3, 4, 6, 1, 2 B. 3, 2, 5, 6, 4, 1
C. 3, 1, 2, 5, 4, 6 D. 1, 5, 4, 6, 2, 3
6. 权值为 {1,2,6,8} 的四个点构成的哈夫曼树的带权路径长度是 ()
A. 18 B. 28 C. 19 D. 29
7. 设有一组初始记录关键字序列为 (34, 76, 45, 18, 26, 54, 92), 则由这组记录关键字生成的二叉排序树的深度为 ()
A. 4 B. 5 C. 6 D. 7
8. 在含有 n 个顶点和 e 条边的无向图的邻接矩阵中, 零元素的个数为 ()
A、 e
B、 $2e$
C、 $n^2 - e$
D、 $n^2 - 2e$
9. 设带有头结点的单向循环链表的头指针为变量为 *head*, 则判断该循环链表为空的条件是 ()

- A、 $head == 0$
- B、 $head \rightarrow next == 0$
- C、 $head \rightarrow next == head$
- D、 $head != 0$

10. 现安排甲、乙、丙、丁、戊 5 名同学参加世博会志愿者服务活动,每人从事翻译、导游、礼仪、司机四项工作之一,每项工作至少有一人参加。甲乙不会开车但能从事其它三项工作,丙、丁、戊都能胜任四项工作,则不同安排工作的方案种数是 ()

- A. 152
- B. 126
- C. 90
- D. 54

11. 假设以行优先顺序存储三位数组 $R[6][9][6]$,其中元素 $R[0][0][0]$ 的地址为 2100,每个元素占 4 个存储单元,则存储地址为 2832 的元素为 ()

- A、 $R[3][3][2]$
- B、 $R[3][3][3]$
- C、 $R[4][3][4]$
- D、 $R[4][3][3]$

12. 12、全局变量 cnt 初始值为 0,则计算 $fib(4)$ 后, cnt 的值为 ()

```

1  int fibo(int n)
2  { // n > 0, cnt = 0;
3      cnt++;
4      if(n == 1 || n == 2) return 1;
5      else return fibo(n - 1) + fibo(n - 2);
6  }
```

- A. 4
- B. 5
- C. 6
- D. 7

13. 13、某台阶共有 10 级,每一步可以跨一级、两级或三级,那么恰好 6 步上完台阶的方法有多少 ()

- A. 120
- B. 90
- C. 78
- D. 144

14. 已知图的邻接矩阵如下,从顶点 0 出发按深度优先搜索遍历的结点序列为 ()

0	1	1	1	1	0	1
1	0	0	1	0	0	1
1	0	0	0	1	0	0
1	1	0	0	1	1	0
1	0	1	1	0	1	0
0	0	0	1	1	0	1
1	1	0	0	0	1	0

- A. 0 1 3 6 5 4 2
- B. 0 1 3 4 2 5 6
- C. 0 1 3 4 2 6 5
- D. 0 2 4 3 1 5 6

15. 由同一组关键字集合构成的各种二叉排序树,下列说法正确的是()

- A、其形态不一定相同,但平均查找长度相同
- B、其形态均相同,平均查找长度也相同
- C、其形态均相同,但平均查找长度不相同
- D、其形态不一定相同,平均查找长度也不一定相同

二、程序阅读

1. 第一题

```
1  #include<bits/stdc++.h>
2  using namespace std;
3
4  int main(){
5      int ax1, ay1, ax2, ay2;
6      int bx1, by1, bx2, by2;
7      cin >> ax1 >> ay1 >> ax2 >> ay2;
8      cin >> bx1 >> by1 >> bx2 >> by2;
9
10     int area1 = (ax2 - ax1) * (ay2 - ay1);
11     int area2 = (bx2 - bx1) * (by2 - by1);
12
13     int width = min(ax2, bx2) - max(ax1, bx1);
14     width = width > 0 ? width : 0;
15     int height = min(ay2, by2) - max(ay1, by1);
16     height = height > 0 ? height : 0;
17
18     cout << area1 + area2 - width * height;
19
20     return 0;
21 }
```

输入满足 $ax1 \leq ax2$, $ay1 \leq ay2$, $bx1 \leq bx2$, $by1 \leq by2$;

其 $-10^4 \leq ax1, ay1, ax2, ay2, bx1, by1, bx2, by2 \leq 10^4$

完成下面的判断题和选择题

1. 程序输出一定为大于0的正整数()

- A、正确
- B、错误

2. 若输入为“0 0 1 1 0 0 1 1”,输出为“1”()

- A、正确
- B、错误

3. 计算过程中不会超出 int 范围。()

- A、正确
- B、错误

4.

A、若输入为“-3 0 3 4 0 -1 9 2”,则输出为()

- A. 51
- B. 55
- C. 45
- D. 42

5. 对于该算法的描述,最准确的是()

- A、计算两个矩形的面积
- B、计算两个矩形的体积
- C、计算两个矩形覆盖的面积
- D、计算两个矩形相交的面积

6. 若输入为“-10 -20 100 200 0 10 20 30”,则输出为()

- A. 24200 B. 25200 C. 20600 D. 22600

2. 第二题

```
01 #include<bits/stdc++.h>
02 using namespace std;
03
04 const int values[] = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1};
05 const char* symbols[] = {"M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"};
06
07 int main(){
08     int num;
09     cin >> num;
10     string s = "";
11     for(int i = 0; i < 13; i++){
12         while(num >= values[i]){
13             num -= values[i];
14             s += symbols[i];
15         }
16         if(num == 0){
17             break;
18         }
19     }
20     cout << s;
21 }
```

输入的数位为小于 4000 的非负整数,完成下面的判断题和选择题。

1. 程序的输出一定为长度大于等于1的字符串()

- A、正确
- B、错误

2. 将第 11 行的 $i < 13$ 改为 $i < 3$,程序输出的结果一定为错误结果。()

- A、正确
- B、错误

3. 当输入为"1900"时,输出为"MCM"。()

- A、正确
- B、错误

4. 该算法第 12 行的 *while* 循环,在单次 *for* 循环中最多循环()次

- A. 2 B. 3 C. 4 D. 5

5. 当输入为” 2145”时,输出为()

- A、*MMCLXL*
- B、*MMCXLV*
- C、*MMCXLVI*
- D、*MMXLV*

6. (4 分) 当输入为“3999”时,输出为()

- A、*MMMCMLXXXIX*
- B、*MMMCMLXXX*
- C、*MMCMXCIX*
- D、*MMCMXCIX*

3. 第三题

```
01 #include<bits/stdc++.h>
02 using namespace std;
03
04 int dis[10010];
05 int main(){
06     int x,y;
07     cin >> x >> y;
08     for(int i = 0;i < 10010;i ++){
09         dis[i] = INT_MAX;
10     }
11     dis[x] = 0;
12     queue<int> q;
13     q.push(x);
```

```

13     while(!q.empty()){
14         //cout << q.size() << endl;
15         int s = q.front();
16         q.pop();
17         if(s == y){
18             cout << dis[y] << endl;
19             break;
20         }
21         if(s+1 < 10010 && dis[s+1] == INT_MAX){
22             dis[s+1] = dis[s] + 1;
23             q.push(s+1);
24         }
25         if(s-1 >= 0 && dis[s-1] == INT_MAX){
26             dis[s-1] = dis[s] + 1;
27             q.push(s-1);
28         }
29         if(s*2 < 10010 && dis[s*2] == INT_MAX){
30             dis[s*2] = dis[s] + 1;
31             q.push(s*2);
32         }
33     }
34
35     return 0;
36 }

```

输入为不超过 10000 的正整数,完成下面的判断题和选择题。

1. 输出的数字同样为一个不超过 10000 的正整数。()
 A、正确
 B、错误
2. 当输入的数字为"1 5"时,队列 q 中同时最多存在的数字为 3 个。()
 A、正确
 B、错误
3. 输出一定大于等于输入数字的差的绝对值()
 A、正确
 B、错误
4. 当两个输入的数字相等时,输出一定为 0.()
 A、正确
 B、错误
5. 当输入的数字为"7 20",输出为()

A. 5

B. 4

C. 6

D. 10

6. 当输入的数字为"1 100",输出为()

A. 100

B. 7

C. 8

D. 10

三、程序完善

1. 第一题

(循环数判断) 判断一个数字是否为循环数

循环数满足:从最高位开始,向右数"该数位"个数字(如果到了最右边就从最左边开始继续数),会停止在一个新的数位并且是一个新的数字,重复这样做,最后经过每个数字一次后能够回到起点。

如果你的数字经过每一个数字后没有回到起点(最高位数)则不是循环数

如果你的数字没有经过每一个数字就已经发生重复,则不是循环数

例如,对于数字3126,从最高位3开始,向右数3个数字到达6,从6开始向右数6个数字(此时会像一个圆环一样继续从3开始数),到达1,此时向右数一个数字到达2,此时向右数2个数字回到起点3. 所以3126是一个循环数。

```
01  #include<bits/stdc++.h>
02  using namespace std;
03
04  int nums[200], nums_rev[200];
05  bool vis[200];
06
07  int main(){
08      int x;
09      cin >> x;
10      int cnt = 0;
11      while(x > 0){
12          nums[cnt] = x % 10;
13          x /= 10;
14      }
15      for(int i = 1; i <= cnt; ++i) nums_rev[i] = nums[cnt - i + 1];
16      bool flag = true;
17      int pos = 1;
18      for(int i = 1; i <= cnt; ++i){
19          if((2) || nums_rev[pos] == 0){
20              flag = false;
21              break;
22          }
23          (3);
24          pos = (4);
25          if(pos == 0) pos = cnt;
26      }
27      if((5)) flag = false;
28      if(flag) cout << "YES";
29      else cout << "NO";
30      return 0;
31  }
```

1. (1) 处应当填写 ()
A、*cnt*
B、*cnt* ++
C、++ *cnt*
D、*cnt* + 1
2. (2) 处应当填写 ()
A、*vis[nums_rev[i]]*
B、*vis[nums_rev[pos]]*
C、!*vis[nums_rev[pos]]*
D、*vis[nums[pos]]*
3. (3) 处应当填写 ()
A、*vis[nums_rev[pos]] = true;*
B、*vis[i];*
C、*vis[nums_rev[i]] = true;*
D、*vis[pos] = true;*
4.
A、(4) 处应当填写 ()
B、(*pos* + *nums_rev[i]*) % *cnt*
C、(*pos* + *nums_rev[pos]*) % *cnt* + 1
D、(*pos* + *nums[pos]*) % *cnt*
E、(*pos* + *nums_rev[pos]*) % *cnt*
5. (5) 处应当填写 ()
A、!*vis[1]*
B、*pos* != 1
C、*pos* != *n*
D、*pos* == 1

2. 第二题

(石子合并) 一条直线上有 N 堆石子, 现要将石子有次序的合并成一堆, 规定每次只能选相邻的 2 堆合并成新的一堆, 并将新的一堆的石子数, 记为该次合并的得分。

思路: 使用 $dp[i][j]$ 表示下标位置 i 到 j 所有元素合并能获得最大得分

考虑区间的拆分进行状态转移

```

01 #include<bits/stdc++.h>
02 using namespace std;
03
04 int a[1024], pre[1024];
05 int dp[1024][1024];
06 int main(){
07     int n;
08     cin >> n;
09     for(int i = 1; i <= n; i++){
10         cin >> a[i];
11         (1);
12     }
13
14     int i, j, k, len;
15     for((2)){
16         for((3)){
17             j = i + len - 1;
18             for(k = i; k < j && k <= n; k++){
19                 dp[i][j] = max(dp[i][j], dp[i][k] + dp[k+1][j] + (4));
20             }
21         }
22     }
23     cout << (5);
24 }

```

1. (1) 处应当填写 ()

- A、 $pre[i] = a[i]$
- B、 $pre[i] = pre[i-1] + a[i]$
- C、 $pre[i] += a[i]$
- D、 $pre[i] = a[i-1] + a[i]$

2. (2) 处应当填写 ()

- A、 $i = n; i \geq 1; i--$
- B、 $i = 1; i \leq n; i++$
- C、 $len = 1; len \leq n; len++$
- D、 $len = n; len \geq 1; len--$

3. (3) 处应当填写 ()

- A、 $i = n; i \geq 1; i--$
- B、 $i = 1; i \leq n; i++$
- C、 $len = 1; len \leq n; len++$
- D、 $len = n; len \geq 1; len--$

4. (4) 处应当填写 ()

- A、 $pre[j] - pre[i-1]$
- B、 $a[k] + a[k+1]$
- C、 $pre[k+1]$
- D、 $pre[j] - pre[i]$

5. (4) 处应当填写 ()

- A、 $dp[n][1]$

B、 $dp[n][n]$

C、 $dp[1][n]$

D、 $dp[1][1]$